

PYTHAI/DELTAVERSE Zero-Knowledge Integration Architecture

Document version: 1.0 — May 9, 2026 **Audience:** Senior software architects (codephreak / Gregory) **License of contained code:** Apache 2.0 with (c) 2026 BANKON — all rights reserved **Tooling baseline:** Python ≥ 3.12 , Solidity $\geq 0.8.26$, Foundry, Podman, OpenBSD vmm, mainnet target.

1. Executive architecture overview

The PYTHAI/DELTAVERSE ecosystem already separates concerns cleanly across three subdomains — `agenticplace.pythai.net` for the agent marketplace, `mindx.pythai.net` for the autonomous cognitive system, and `bankon.pythai.net` for identity and payment — and across two chain layers — Algorand as the constitutional layer and an EVM economic layer (Ethereum mainnet primary, Polygon, Arbitrum, Base, Moonbeam supporting, with the Arc/Circle Chain ID 5042002 slot reserved). **Zero-knowledge cryptography is the connective tissue that lets these subsystems compose without leaking state across trust boundaries.** Every cross-domain interaction in the ecosystem can be reduced to one of four ZK questions: *who is acting* (identity attestation), *can they pay for it* (payment privacy), *was the work actually done correctly* (compute integrity), and *did the collective sanction the result* (governance). The unified architecture answers all four with proofs whose verifiers live on-chain on either Algorand or an EVM destination, and whose provers run client-side, in TEE coprocessors, or on decentralized prover networks (Boundless, Succinct Prover Network, Polyhedra Proof Cloud, Aligned Layer).

The four integration layers are not independent silos; they compose into a single causal chain. **An identity attestation feeds into a payment authorization, which gates a compute integrity proof, which produces a governance-relevant attestation.** Concretely: a mindX agent on AgenticPlace must (1) prove via a Fides-anchored reputation predicate that it is above a service threshold and bound to a specific aGLM checkpoint and ERC-7857 INFT, then (2) accept a confidential x402 V2 payment whose payer has demonstrated solvency and Sybil-resistance without revealing balances, then (3) execute inference under a TEE+ZK hybrid attestation that produces a succinct receipt over the (input commitment, output, checkpoint hash) triple, and finally (4) commit that receipt into a Tabularium audit log whose Curia projection is verifiable by regulators and Senatus voters under selective disclosure. Each step's output is the next step's witness. Because every step terminates in a SNARK whose verifier is a deterministic constant-gas Solidity or algopy contract, the entire chain is composable in a way no traditional API stack can match: **a single recursive proof can attest to all four layers having been honored for a single agent action.**

The first-principles justification for proof-system choice per layer follows the dominant economic constraint at that layer. **Identity proofs are small, frequent, and need cheap on-chain verification**, so they pin to **Groth16 over BN254** with per-circuit ceremonies (~200-byte proofs, ~250k gas verification including calldata), or to **PLONK/UltraPlonk** over a universal SRS where circuit churn is expected. Semaphore v4 (LeanIMT, EdDSA identity) provides the canonical group-membership primitive; BBS#/BBS+ over BLS12-381 covers the W3C Verifiable Credential surface; EAS (Ethereum Attestation Service) at attest.org provides the on-chain schema registry, and its 2025 ZK Playbook standardizes SP1→Groth16 wrapping for selective-disclosure flows. **Payment privacy needs higher prover throughput and shielded-pool semantics**, so it pins to **Halo2-KZG (Railgun-class)** for shielded ERC-20 over USDC/IDC, **Privacy Pools (Oxbow, Buterin et al. 2023)** with ASP screening for compliance-aware flows, and a custom **circum-Groth16 nullifier circuit** piggybacked onto the x402 V2 PAYMENT-SIGNATURE header to give servers proof-of-payment without revealing the sender, all anchored on-chain by EIP-3009 or Permit2 settlement and on the Algorand side by AVM v12 atomic-group transactions with falcon_verify for post-quantum payer auth. **Compute integrity at the LLM tier exceeds pure-ZKML feasibility for any model larger than ~7B parameters**, so the architecture mandates a **TEE+ZK hybrid**: Intel TDX + NVIDIA H100/H200 confidential mode (per OG TeeML and Phala dstack) produces a hardware-signed DCAP attestation, and a RISC Zero or SP1 zkVM circuit re-verifies that attestation chain on-chain, wrapped to Groth16 for ~275–300k gas final verification (Automata DCAP Attestation v1.1 pattern). Sub-7B specialist tiers (aGLM-BANKON-edge, aGLM-BANKON-mid, aGLM-BANKON-specialist-code) become candidates for **pure ZKML** via Lagrange DeepProve or ZKTorch (sumcheck + logup-GKR; GPT-J 6B in ~20 minutes). **Governance and bridging require the strongest cryptoeconomic guarantees and the smallest verifier surface**, so they pin to **SP1 zkVM** for bridge proofs (~275k gas final verifier on EVM via Groth16 wrap; Telepathy/Helios pedigree, secures >\$1B TVL across Gnosis OmniBridge, IBC Eureka, OP Succinct), **Algorand State Proofs** with their forthcoming SNARK derivative for the constitutional → economic direction, and **MACI-style coordinator-aggregated voting** with anonymous Semaphore eligibility for the DAIO Boardroom and War Council. Xn--2-umb + 12

The ecosystem's nine BANKON Solidity contracts map to specific layers as follows: **Genius** holds the canonical registry of agent and human principals, and is touched by every ZK identity flow as the source of truth for binding (eth_address, ENS subname, INFT tokenId, aGLM checkpoint hash). **BonaToken** is the BONAFIDE-branded ERC-20 used as the gas-of-reputation; its transfers can route through a Railgun-style shielded pool wrapper for confidential reputation-weighted flows. **Tabularium** is the append-only audit log; every ZK verification result emits a Tabularium event that becomes the input to Curia projections. **Fides** holds reputation scores; ZK predicates over Fides ("score > X") use a Pedersen commitment over the Fides storage slot plus a Bulletproof or Groth16 range proof. **SponsioPactum** is the bonded-promise contract; its bonded-action gating uses ZK proofs of solvency. **Censura** is the moderation/slashing gate; it consumes anonymous reports backed by Semaphore proofs of eligibility. **Senatus** is the governance contract; its vote tally is a MACI batch with ZK proofs of correct aggregation. **Tessera** is the credential/badge issuer; each tessera is an EAS attestation whose ZK presentation goes through

the BBS#-on-BLS12-381 path. **Curia** is the regulator-facing read-side; it consumes Tabularium events and exposes selective-disclosure ZK queries. The Algorand (`alcopy`) ports of these contracts mirror the same logic over AVM v12 opcodes, using (`mimc`) for SNARK-friendly hashing, (`ec_pairing_check`) (BN254/BLS12-381) for verifier inner loops, (`falcon_verify`) for PQ payer auth, and box storage for nullifier sets. (`Algorand`)

Three architectural constants govern everything. **First, immutability of verifier contracts.** A SNARK verifier embeds a verification key (VK) that is a commitment to the circuit; an upgradeable verifier is a malicious-VK-injection vulnerability. The architecture therefore deploys verifiers as immutable contracts and routes through a (`VerifierRegistry`) (Railgun v3 pattern) that is itself governed by a 2-of-N Gnosis Safe + 48h (`TimelockController`), with the registry mapping (`circuitId → verifierAddress`). New circuits get new verifier deployments; old proofs remain verifiable on old verifiers; nothing is ever upgraded in place. **Second, the Pontifex xERC20 sovereignty layer (EIP-7281) governs cross-chain token semantics.** All BONAFIDE/IDC/BonaToken cross-chain transfers must route through xERC20 mint/burn paths with per-bridge rate limits (`mintingMaxLimitOf`, `burningMaxLimitOf`) resetting every 24h), and the ZK extensions to xERC20 — confidential mint/burn proofs — become a privacy-preserving overlay rather than a parallel system. **Third, ENS NameWrapper fuse permanence is non-revocable.** Once (`CANNOT_UNWRAP`), (`CANNOT_BURN_FUSES`), (`CANNOT_TRANSFER`), and (`CANNOT_SET_RESOLVER`) are burned on a (`bankon.eth`) subname, the binding (subname, owner, resolver) is locked until expiry, providing a cryptographically-anchored anchor for ZK selective-disclosure proofs. The architecture exploits this by including (`(subname_namehash, fuse_state, expiry)`) as public inputs to identity circuits; once (`PARENT_CANNOT_CONTROL`) is burned, parent governance cannot rewrite the binding either. (`ENS`) (`ENS`)

Recursive aggregation is the integration glue. The architecture's canonical **proof composition pattern** is: leaf circuits (identity, payment, compute, governance) emit Halo2 / Plonk / Groth16 / SP1-Groth16 proofs to their respective chains; per-chain aggregator services (Aligned Layer for EVM, a custom Algorand State Proof aggregator for AVM) batch these into per-chain root proofs; a **master recursive aggregator running in SP1 Hypercube** (achieving 99.7% of mainnet blocks proven in <12s on 16 RTX 5090 GPUs as of November 2025) folds per-chain roots into a single **Pythai Universal Receipt (PUR)** committed to Ethereum L1. The PUR becomes the canonical artifact for cross-domain auditing: regulators verifying an agent action only need to verify one Groth16 wrap (~275-300k gas) and trust the recursive structure underneath. (`Succinct`)

Three decision criteria govern the TEE-vs-ZK-vs-hybrid choice per subsystem. **Use TEE-only** when latency dominates and on-chain verification is rare (mindX flagship/specialist-think tier inference at >7B params; cost: silicon trust + cloud trust). **Use ZK-only** when the workload fits a circuit and on-chain settlement requires no off-chain trust (payment shielding, identity predicates, governance tally; cost: prover time). **Use hybrid (TEE-attests-then-ZK-wraps)** when both are needed: the heavy compute runs in TEE for speed, the attestation signature is verified inside a small zkVM circuit, and the final on-chain verifier burns ~300k gas instead of running

ECDSA-P-256 chain validation natively. This is the canonical pattern for ERC-7857 INFT transfers, aGLM flagship inference attestation, and OG TeeML-backed mindX endpoints.

The remainder of this document specifies each layer in detail, the proof systems in tradeoff form, the chain-by-chain mapping, the Foundry test scaffolding, the algopy verifier contracts, the parsec-wallet ZK extension, the mindX ZKML integration, the security analysis, the deployment roadmap, and the current state-of-the-art survey grounded in primary sources from eprint.iacr.org, arxiv.org, the Ethereum and Algorand EIP/ARC indices, and vendor engineering blogs from the November 2025 - May 2026 window.

2. Identity / Attestation ZK layer — deep dive

The identity layer's atomic unit is a **Genius principal**: a row in the `Genius` contract binding an Ethereum address (or Algorand address), an ENS subname under `bankon.eth`, an optional ERC-7857 INFT tokenId, an optional aGLM checkpoint hash, and a Fides reputation slot. Every ZK identity proof in the ecosystem is a predicate over some subset of this tuple, and the design problem is which subset to expose as public inputs and which to hide as witnesses.

2.1 BANKON ENS subname privacy with NameWrapper fuses

ENS NameWrapper fuses provide a 96-bit permission field where burned bits cannot be unburned until expiry, and the subname registrar at `bankon.eth` is configured to burn `PARENT_CANNOT_CONTROL`, `CANNOT_UNWRAP`, `CANNOT_BURN_FUSES`, `CANNOT_TRANSFER`, `CANNOT_SET_RESOLVER`, and `CANNOT_CREATE_SUBDOMAIN` on issuance, leaving only `CAN_EXTEND_EXPIRY` for KeeperHub renewals. **The subname-ownership selective-disclosure circuit** proves: (1) knowledge of a private key whose public key resolves to the address bound to a subname `<label>.bankon.eth`, (2) that subname's NameWrapper fuse state has the locked-down bits set, (3) that the subname has not expired, all without revealing `<label>`.

Implementation uses a **circom-Groth16** circuit with public inputs `(parent_namehash = namehash(bankon.eth), root = NameWrapper.tokenURI commitment, current_block, fuse_mask)` and witnesses `(label, owner_pubkey, signature, merkle_path_in_namewrapper)`; the circuit constrains `namehash(label, parent_namehash)` to match a leaf in the NameWrapper Merkle commitment, asserts `(stored_fuses & fuse_mask) == fuse_mask`, and checks an EdDSA signature over a challenge. The verifier on Ethereum mainnet costs ~250k gas; on Polygon, Arbitrum, Base, and Moonbeam the same Groth16 verifier deploys at the same CREATE2 address. [GitHub](#) [npm](#)

```

// SPDX-License-Identifier: Apache-2.0
// (c) 2026 BANKON – all rights reserved
pragma solidity ^0.8.26;

import {IGroth16Verifier} from "../interfaces/IGroth16Verifier.sol";

contract BankonSubnameDisclosureGate {
    IGroth16Verifier public immutable verifier;
    bytes32 public immutable bankonEthNode; // namehash("bankon.eth")
    uint96 public immutable requiredFuses; // CU|CBF|CT|CSR|CCS|PCC

    mapping(bytes32 => bool) public usedNullifiers;

    event SubnamePresented(bytes32 indexed nullifier, address indexed gatedAction, u

    constructor(address _verifier, bytes32 _node, uint96 _fuses) {
        verifier = IGroth16Verifier(_verifier);
        bankonEthNode = _node;
        requiredFuses = _fuses;
    }

    function present(
        uint[2] calldata pA, uint[2][2] calldata pB, uint[2] calldata pC,
        uint[5] calldata publicInputs // [node, root, block, fuseMask, nullifier]
    ) external {
        require(publicInputs[0] == uint256(bankonEthNode), "wrong parent");
        require(publicInputs[3] == uint256(requiredFuses), "wrong fuse mask");
        require(block.number - publicInputs[2] < 256, "stale block");
        bytes32 nf = bytes32(publicInputs[4]);
        require(!usedNullifiers[nf], "nullifier reused");
        require(verifier.verifyProof(pA, pB, pC, publicInputs), "invalid proof");
        usedNullifiers[nf] = true;
        emit SubnamePresented(nf, msg.sender, block.timestamp + 1 hours);
    }
}

```

2.2 Fides reputation predicates

Fides stores per-principal reputation as a `uint256` score plus a Merkle root of the reputation event log (each event being a tuple `((actor, target, delta, evidence_hash, timestamp))`). The **reputation-threshold circuit** proves `score >= threshold` without revealing the score, and additionally proves the score is consistent with the Tabularium event log through a Merkle-

inclusion check. The architecture supports three flavors. **Flavor A: Semaphore-style group membership.** Principals with score above tiers (`bronze`, `silver`, `gold`, `platinum`) are added to per-tier Semaphore groups (LeanIMT, EdDSA identity, depth 32); proving "I am at least silver" is a Semaphore v4 proof against the silver group's Merkle root. This is cheap (~250k gas verifier on EVM) and Sybil-safe via per-action external nullifiers, but the tiers are coarse-grained. **Flavor B: BBS+/BBS# anonymous credentials over BLS12-381.** Fides issues a BBS+ signature `((σ, A, e, s))` over the attribute vector `((principal_id, score, timestamp, nonce))`; the holder generates a presentation proof that selectively discloses some attributes (e.g., reveal `timestamp`) and proves predicates on others (`score >= 700`) without revealing them. BBS+ presentations are ~600 bytes; on-chain verification requires BLS12-381 pairing precompiles (EIP-2537, available on Ethereum mainnet since Pectra) and costs ~120k-180k gas. **Flavor C: zk-attestations through EAS.** Fides emits an EAS attestation per reputation event using the schema `((bytes32 principal, int256 delta, bytes32 evidenceHash, uint64 ts))`; a presenter uses the EAS ZK Playbook (SP1 → Groth16 wrap) to prove "the sum of disclosed deltas across attestations whose principal-hash matches my Semaphore identity exceeds X" without revealing the individual attestations. ([GitHub](#))

The canonical Fides predicate gate dispatches to one of the three based on the use case: gating votes prefers Flavor A (cheap and tier-coarse is fine), gating cross-chain reputation portability prefers Flavor B (because BBS# is the W3C VC-DI standard), and audit-trail-grounded predicates prefer Flavor C.

2.3 Agent verification — binding mindX agent ↔ aGLM checkpoint ↔ INFT

The **agent binding circuit** proves: "I am the holder of an ERC-7857 INFT whose `intelligentDataOf(tokenId).dataHash` equals `H_checkpoint`, and `H_checkpoint` equals the Merkle root of an aGLM checkpoint tier file in the registry, and I have the active TEE attestation receipt over `((H_checkpoint, current_epoch_nonce))`." Public inputs: `((tokenId, checkpointTierId, epochNonce, attestationDigest))`. Witnesses: `((holder_signature_over_nonce, dataHash_preimage_layout, attestation_full_quote_inside_zkvm))`. Implementation runs the DCAP attestation chain verification inside SP1 (the canonical Automata pattern, ImageID `6f661ba5...bc807` per Automata Testnet), wraps to Groth16 (~275k gas), and additionally constrains the INFT data hash. The architecture mandates this circuit as a precondition for any agent appearing on AgenticPlace; the listing contract calls `verifier.verify(proof, publicInputs)` and emits `AgentBound(tokenId, checkpointTierId, presenter)` into Tabularium. ([Ethereum](#)) ([ethereum](#))

2.4 Sybil resistance for AgenticPlace

The marketplace requires per-action Sybil-resistance for posting listings, leaving reviews, and casting reputation votes. The architecture composes **World ID 4.0** (orb-based proof of personhood with the new threshold-OPRF nullifier protocol, replacing the legacy single-secret model) and **Human Passport** (formerly Gitcoin Passport, acquired by Holonym in 2025, ~2M

users, 35M+ credentials) into a single AgenticPlace (PoH) contract. The (PoH) contract accepts either a Semaphore v4 proof against the World ID Merkle root with a per-action external nullifier, or an SP1-wrapped zk-Email/zkPassport/government-ID Holonym credential. Users meeting either path receive a one-time (humanity_token) that is itself a nullifier-bounded proof reusable for the action's lifetime. The architecture defaults to ANY-of (either path is sufficient) for low-stakes actions and ALL-of (both paths) for high-stakes actions like listing a flagship-tier agent. (Cointelegraph)

2.5 Censura / Senatus voter eligibility with reputation-weighted ballots

Senatus governance uses a MACI-style flow: the coordinator collects encrypted ballots, batches them, and produces a ZK proof of correct tally. **Voter eligibility** is a Semaphore v4 proof against the Senatus voter-set Merkle root, with the external nullifier set to (proposalId, ballotEpoch). **Reputation weighting** is the difficult part: a voter's weight equals the floor-log of their Fides score, and the tally circuit must aggregate ($\sum \text{weight}_i \cdot \text{vote}_i$) without revealing per-voter weights. The architecture solves this with a **Pedersen commitment over weights**: each voter's identity commitment in the Senatus group includes a Pedersen-blinded weight ($C_i = g^{\{w_i\}} h^{\{r_i\}}$); the ballot encrypts (vote, w_i , r_i) to the coordinator's ElGamal key; the coordinator's tally circuit (compiled in Noir under UltraHonk for prover speed) proves ($\sum w_i \cdot \text{vote}_i$) over the homomorphic sum, and reveals only the aggregate. Censura moderation actions reuse the same eligibility circuit but with a higher tier threshold; slashing decisions go through a separate Boardroom multisig path.

2.6 EAS schema definitions

text

```
// Bankon Subname Binding
schema: "bytes32 subnameNamehash, address owner, uint96 fuseState, uint64 expiry, bytes32 evidenceHash, uint64 ts, uint8 tier, bytes32 trainerAttestation, bytes32 dataHash, uint64 expiresAt, uint64 revocableAt"
revocable: false

// Fides Reputation Event
schema: "bytes32 principalId, int256 delta, bytes32 evidenceHash, uint64 ts, uint8 tier, bytes32 trainerAttestation, bytes32 dataHash, uint64 expiresAt, uint64 revocableAt"
revocable: true (only by Curia)

// aGLM Checkpoint Attestation
schema: "bytes32 checkpointHash, uint8 tier, bytes32 trainerAttestation, bytes32 dataHash, uint64 expiresAt, uint64 revocableAt"
revocable: false

// Tessera Credential
schema: "bytes32 holder, bytes32 credentialType, uint64 issuedAt, uint64 expiresAt, uint64 revocableAt"
revocable: true
```

All four schemas register at attest.org against the Ethereum mainnet, Base, Optimism, Arbitrum, Scroll, and Linea EAS deployments.

2.7 parsec-wallet identity flow

The parsec-wallet identity flow is: (1) on first launch, the wallet derives a cypherpunk2048 master seed; (2) from this seed it derives a Semaphore identity, a BBS#/BLS12-381 keypair, an EdDSA NameWrapper-binding key, and an Ethereum/Algorand spending key; (3) it scans EAS attestations, BANKON Tabularium events, and ENS NameWrapper events relevant to its public addresses to build a local credential store; (4) on a presentation request, it generates the appropriate proof in WASM (circom-snarkjs for Semaphore, bb.js for Noir/Honk circuits, or a remote SP1 proof through a configured prover endpoint). Prover times are 1–5s for Semaphore, 5–60s for Noir/Honk circuits in browser, and 30–600s for SP1 zkVM circuits remotored to Boundless or Succinct Prover Network.

3. Payment privacy ZK layer — deep dive

The payment layer must answer: *can we attach a ZK proof to an x402 V2 HTTP request such that the resource server verifies payment without seeing the underlying transaction, on both Ethereum-class chains and Algorand, with USDC, BONAFIDE BonaToken, and IDC stablecoin all routed through the same primitives?*

3.1 x402 V2 header schema extension for ZK proofs

The x402 V2 protocol places a base64-encoded `PaymentPayload` in the `PAYMENT-SIGNATURE` header. The architecture extends the payload with a `zk` block: [GitHub +2](#)

```
json
```



```

{
  "x402Version": 2,
  "scheme": "exact-zk",
  "network": "eip155:8453",
  "payload": {
    "settlement": {
      "asset": "eip155:8453/erc20:0x833589fCD6eDb6E08f4c7C32D4f71b54bdA02913",
      "shieldedPoolRoot": "0xa1b2...",
      "nullifier": "0xc3d4...",
      "outputCommitment": "0xe5f6..."
    },
    "proof": {
      "system": "groth16-bn254",
      "verifierId": "0x...registry-id...",
      "publicInputs": ["0x...", "0x...", "0x...", "0x..."],
      "proofBytes": "base64..."
    }
  },
  "extensions": {
    "x402-zk-version": 1,
    "facilitatorPolicy": "verify-only"
  }
}

```

The `exact-zk` scheme is registered at the facilitator with `network`-bound verifier registry addresses. The facilitator's `/v2/x402/verify` endpoint computes `verifierRegistry.lookup(verifierId)` and calls `verifier.verifyProof(...)`; if the proof is valid and the nullifier is unspent in `ShieldedPool.usedNullifiers`, it returns success without performing on-chain settlement (the user has already settled into the shielded pool out-of-band, and the proof binds the spend to this specific resource URL via the public inputs). For atomic-settlement variants, the facilitator's `/v2/x402/settle` endpoint additionally calls `ShieldedPool.spend(nullifier, outputCommitment, recipient, amount, proof)` which transfers value to `payTo` and burns the nullifier in one transaction. [Coinbase](#) [GitHub](#)

3.2 parsec-wallet ZK proof generation flow

Parsec-wallet's cypherpunk2048 key handling extends to a fully Zcash-Sapling-style spending/viewing-key separation. From the master seed, the wallet derives:

- `ask` (Spending Authorization Key) — required for spend authorization, kept hot only at signing time, optionally backed by a YubiKey or hardware enclave.

- `(nsk)` (Nullifier Secret Key) — derived from `(ask)`, used to compute nullifiers as `(nf = PRF(nsk, note_commitment))`.
- `(ovk)` (Outgoing Viewing Key) — decrypts memos on outgoing notes, shareable with auditors.
- `(ivk)` (Incoming Viewing Key) — derives diversified addresses, scans incoming notes; shareable with parsing services.
- `(dk)` (Diversifier Key) — generates many diversified payment addresses from one viewing key.

Stealth addresses follow ERC-5564 with the canonical singleton announcer at

`0x55649E01B5Df198D18D95b5cc5051630cfD45564` and meta-address registry at

`0x6538E6bf4B0eBd30A8Ea093027Ac2422ce5d6538`. Parsec-wallet publishes its stealth meta-

address `(st:eth:<spendingPubKey><viewingPubKey>)` to ERC-6538 once and uses 1-byte view tags for fast scanning. [GitHub](#) [GitHub](#)

The proof generation flow on a payment request: (1) wallet receives 402 response with `(PaymentRequired)` header; (2) wallet selects an unspent shielded note from its local store; (3) wallet computes `(nf = PRF(nsk, cm))`, generates a new output commitment `(cm_out)` to the recipient's stealth address (or to a refund self-address if amount > price), and a Pedersen-balance-equation witness; (4) wallet generates a Groth16 proof in WASM (circom-snarkjs, ~5-30s on consumer hardware for a 2-input 2-output Sapling-style spend circuit; using `(ezkl-gpu)` accelerated browser path drops this to 1-5s on machines with GPU); (5) wallet base64-encodes the payload and retries the HTTP request with `(PAYMENT-SIGNATURE)`; (6) wallet listens for `(PAYMENT-RESPONSE)` for settlement confirmation.

3.3 Algorand x402 transaction structure with `(falcon_verify)`

The Algorand x402 path uses the existing `(algorand:SG01...0iI=)` mainnet CAIP-2 ID and atomic transaction groups (≤ 16 elements). The architecture extends this with a ZK-payment scheme: (1) the payer constructs a group `([payment_to_pool_app, app_call_with_proof, optional_falcon_sig_logicsig_account])`; (2) the `(app_call_with_proof)` includes the ZK proof bytes split across `(BoxRef)` storage and a header in `(Note)`; (3) the algopy contract verifies the proof using `(ec_pairing_check BN254g1=3600+90/32B)` opcodes pooled across the group's 700·N opcode budget; (4) on success, the contract transfers from the shielded pool's escrow address to the resource server's address. For PQ-safe payers, the LogicSig path adds `(falcon_verify(challenge, sig, pk))` (AVM v12, PR #5599) so the payer's authorization remains lattice-secure; this aligns with the cypherpunk2048 spec's PQ posture. [GitHub + 2](#)

3.4 EVM-side privacy pool integration

The architecture deploys two parallel EVM privacy paths and selects per use case. **Railgun-class shielded pool** (Groth16, BN254, 54 circuits parameterized by UTXO input/output count, ~\$70M historical TVL, audited, Proof-of-Innocence enabled) is the default for confidential BONAFIDE

BonaToken transfers and IDC stablecoin movement — full shielding, no compliance overlay required because BANKON-internal flows are policed by Fides and Curia rather than by an external ASP. **Privacy Pools (Oxbow, Buterin et al. 2023)** with ASP screening is used for USDC inbound/outbound at the BANKON $\leftrightarrow$ USD perimeter — users prove their withdrawal belongs to an approved set without revealing which deposit, providing the regulatory-compliance equilibrium that lets BANKON serve institutional flows. Both pools sit behind a single `ShieldedPaymentRouter` Solidity contract that the x402 facilitator dispatches to based on `(asset, region_policy)`. `Flashift` `Bankless`

3.5 SHAMBA LUV emotonomics protocol

SHAMBA LUV's emotional/social signal aggregation reduces to **anonymous bounded-value sum aggregation with ZK range proofs**. Users submit per-event signals $s_i \in [0, S_{\max}]$, where each s_i is a quantized affective valence/arousal pair; each submission is a Pedersen commitment $C_i = g^{\{s_i\}} h^{\{r_i\}}$ plus a Bulletproof range proof (~600 bytes, ~150k gas verification on a Solidity Bulletproofs verifier or ~30k gas on a Plonky3 sumcheck verifier); double-submission is prevented by a Semaphore-v4 nullifier scoped to the event ID. The aggregator multiplies $C_{\text{total}} = \prod C_i$ (homomorphic sum) and reveals $\sum s_i$ along with a discrete-log opening. For higher throughput the architecture optionally swaps Pedersen for **Prio-style two-server secret sharing** (deployed in Mozilla Telemetry at scale, 100× faster than ZK-only approaches), where two non-colluding aggregator servers each see only one share of each s_i , and each share carries a SNIP attesting to its validity range. `ResearchGate` `Semantic Scholar`

3.6 BONAFAIDE BonaToken transfer privacy and IDC confidential transactions

BonaToken and IDC are deployed as xERC20 (EIP-7281) tokens across all primary EVM chains, with `mintingMaxLimit0f` set per-bridge per 24h and Pontifex enforcing the issuer-controlled bridge whitelist. The ZK overlay wraps both inside the same `ShieldedPaymentRouter`: a BonaToken `Lockbox` (per EIP-7281) accepts canonical deposits and mints into the shielded pool's Merkle tree; transfers within the shielded set are ZK-private; withdrawals burn from the tree and unlock from the Lockbox. IDC follows the same structure. Cross-chain shielded transfer uses a Pontifex xERC20 mint on the destination chain whose authorization is itself a ZK proof of a corresponding burn on the source chain (the SP1 cross-chain proof from §5.1).

3.7 End-to-end payment-with-privacy sequence

A user `Alice` wants to access `mindx.pythai.net/v1/inference?model=aGLM-BANKON-flagship` for 0.05 USDC. (1) Alice's parsec-wallet GETs the URL; mindX returns 402 with `PAYMENT-REQUIRED` listing accepted schemes including `exact-zk` over `eip155:8453/erc20:USDC` to the BANKON shielded pool. (2) Wallet checks if Alice has shielded USDC; if not, it first deposits 1 USDC into Privacy Pools through Kohaku-style flow with an ASP attestation. (3) Wallet selects a 0.06 USDC note (small over-pay with refund), computes `nf`, builds a 2-in-2-out Groth16 proof binding the spend to the resource URL via public inputs, and packages the `PAYMENT-SIGNATURE`

header. (4) Wallet retries the GET with the header. (5) mindX's x402 middleware extracts the payload, posts to `(https://x402.bankon.eth/v2/x402/verify)` (the BANKON-operated facilitator). (6) Facilitator looks up the verifier registry, calls `(Groth16Verifier.verifyProof(...))` (~250k gas in eth_call simulation, free), checks the nullifier is unspent, returns 200. (7) mindX's middleware proceeds to inference. (8) mindX returns 200 with `(PAYMENT-RESPONSE)`. (9) The facilitator settles asynchronously by submitting the spend to `(ShieldedPool.spend(...))` on Base, which burns the nullifier on-chain and transfers 0.05 USDC to mindX's payTo address. The merchant never sees Alice's address; Alice never reveals her balance; the Privacy Pools ASP guarantees compliance. Total wall-clock: ~3-8 seconds for proof generation, ~1 second for HTTP roundtrip, ~12 seconds for Base settlement (asynchronous). `(GitHub + 2)`

4. Compute integrity ZK layer — deep dive

4.1 aGLM inference attestation per checkpoint tier

The five aGLM tiers map to five distinct attestation strategies:

- **aGLM-BANKON-edge** (≤ 1 B params, INT8-quantized, Qwen3.5-1B base): pure ZKML via **Lagrange DeepProve** or **JoltAtlas**. DeepProve's sumcheck + logup-GKR proves transformer inference at 54-158 $\times$ the speed of EZKL/Halo2; JoltAtlas drops the RISC-V CPU emulation tax by mapping ONNX tensor graphs directly to sumcheck-based lookups. Expected per-inference proof time: 30s-3min on a single RTX 5090; proof size <200KB; verifier gas ~300k after Groth16 wrap. Use case: AgenticPlace marketplace listings where buyers need to verify the seller's quoted output really came from a specific edge-tier checkpoint. `(Cryptonewsnavigator + 2)`
- **aGLM-BANKON-mid** (~3-7B, INT4 weights with INT8 activations, Qwen3.5-7B base): ZKML via **ZKTorch** (parallelized folding, basic-block accumulation; GPT-J 6B in ~20 minutes on 64 threads) or **zkLLM** (CUDA-parallelized; OPT-13B and LLaMa-2-13B in <15 minutes; tlookup + zkAttn). Per-inference proof time: 5-30 minutes on a 4 $\times$ H100 cluster; proof size ~200KB; verifier gas ~300k. `(Substack + 3)`
- **aGLM-BANKON-flagship** (>20B, the full system-of-intelligences tier): **TEE+ZK hybrid mandatory**. Inference runs in 0G TeeML (Intel TDX + H100/H200 confidential mode, OpenAI-compatible API at `(pc.0g.ai)`); the TEE produces a DCAP attestation chain over `(image_hash, input_commitment, output_hash, model_weights_hash)`; an SP1 zkVM circuit re-verifies the DCAP chain (PCK Cert $\rightarrow$ Processor/Platform CA $\rightarrow$ Intel Root, plus TCB Info, QE Identity, RIM, OCSP staple) and wraps to Groth16 (~275-300k gas). The Phala "Private Proving" pattern places the SP1 prover *inside* a separate TEE so the prover never sees the model weights or user data, with <20% TEE overhead. `(WEEX + 5)`

- **aGLM-BANKON-specialist-code** (~7B, code-fine-tuned LoRA over Qwen3.5-7B): **TEE for inference + zkLoRA for fine-tuning attestation**. zkLoRA (Liao et al. 2025) proves LoRA adapter updates end-to-end far cheaper than full training proofs. (Emergent Mind)
- **aGLM-BANKON-specialist-think** (~13B-70B, reasoning-specialized): **TEE+ZK hybrid identical to flagship**, with the addition of a Tabularium projection of the reasoning trace's hash for after-the-fact selective-disclosure auditing.

The mindX API at `mindx.pythai.net/v1` exposes:

- `POST /v1/inference` — runs inference, returns `(output, attestation_receipt_id)`.
- `GET /v1/attestations/:id` — returns the SP1 receipt and Groth16 wrap proof.
- `POST /v1/attestations/:id/verify` — server-side verification helper for clients without full ZK stacks.
- `POST /v1/proof-of-checkpoint` — given a tokenId, returns the agent-binding circuit proof.

4.2 Hybrid TEE+ZK fallback formal pattern

```
python
```

```

# (c) 2026 BANKON – all rights reserved
# Apache-2.0
# mindx/inference/hybrid_attestor.py
from __future__ import annotations
from dataclasses import dataclass
from hashlib import sha256
import asyncio

@dataclass(slots=True, frozen=True)
class InferenceWitness:
    checkpoint_hash: bytes
    input_commitment: bytes
    output: bytes
    nonce: bytes

@dataclass(slots=True, frozen=True)
class HybridReceipt:
    tee_quote: bytes          # Intel TDX DCAP quote (~5KB)
    sp1_groth16: bytes        # SP1-generated Groth16 wrap (~256B)
    public_inputs: list[int]
    witness_digest: bytes

class HybridAttestor:
    def __init__(self, tee_endpoint: str, sp1_prover_endpoint: str) -> None:
        self.tee_endpoint = tee_endpoint
        self.sp1_prover_endpoint = sp1_prover_endpoint

    async def attest(self, witness: InferenceWitness) -> HybridReceipt:
        digest = sha256(
            witness.checkpoint_hash + witness.input_commitment +
            witness.output + witness.nonce
        ).digest()
        tee_quote = await self._tee_sign(digest)
        sp1_proof, public_inputs = await self._sp1_wrap(tee_quote, digest)
        return HybridReceipt(
            tee_quote=tee_quote,
            sp1_groth16=sp1_proof,
            public_inputs=public_inputs,
            witness_digest=digest,
        )

```

```
async def _tee_sign(self, digest: bytes) -> bytes: ...
async def _sp1_wrap(self, quote: bytes, digest: bytes) -> tuple[bytes, list[int]]
```

4.3 ERC-7857 evolution proof

ERC-7857 (Wu, Zeng, Wu, Heinrich; OG Labs; Draft 2025-01-02) defines

`verifyTransferValidity(TransferValidityProof[]) → TransferValidityProofOutput`. The architecture's evolution-proof construction adds a fifth field to the standard's ProofOutput: `lineage_root`, the Merkle root over all prior `(oldDataHash, newDataHash, transferTimestamp)` triples for this tokenId, allowing any future verifier to prove the agent's evolution from genesis without revealing intermediate states. The on-chain INFT contract maintains `mapping(uint256 tokenId => bytes32 lineageRoot)`; each successful transfer updates the root via a ZK proof that the new root is the previous root extended with the current transfer triple. Implementation uses an SP1 zkVM circuit that consumes the prior lineage root, the current transfer's three fields, and outputs the new root — wrapped to Groth16 for ~275k gas verification. (ethereum)

4.4 mindX Knowledge Catalogue verifiable queries

The mindX append-only memory log has CQRS projections into Kuzu (graph), Qdrant (vector), and Meilisearch (BM25). The architecture's verifiable-query strategy is differentiated per backend.

Vector queries (Qdrant): Use V3DB (Qiu, Qu, Zhang, Yuan 2025; Plonky2 implementation, 22× faster proving than naive Halo2 baseline; ms-level verification). The corpus snapshot is committed; queries return top-k payloads plus a Plonky2 proof that the result matches the committed snapshot under IVF-PQ retrieval semantics. The `mindx.pythai.net/v1/search/vector` endpoint returns `(results, plonky2_proof, snapshot_commitment)`; clients verify locally or via the mindX verifier endpoint. (arxiv) (arxiv)

Graph queries (Kuzu): Research-frontier; the architecture commits the graph as a Verkle-trie of nodes and adjacency commitments (SonicDB-S6-style batched commitments) and provides Merkle inclusion proofs for path queries up to depth 3. Beyond depth 3 the architecture falls back to TEE-attested query execution (Phala dstack runs the Kuzu engine under TDX, signs the result; SP1 wraps the attestation).

BM25 queries (Meilisearch): No production ZK system proves BM25 today. The architecture uses TEE+ZK hybrid: Meilisearch runs in a TDX CVM, signs the result, SP1 wraps the attestation. A research track funded under BANKON's grant program targets a circom-based BM25 proof for top-3 retrieval over an inverted-index commitment as a 2027 deliverable.

Append-only log integrity: The memory log itself is committed as a Merkle root updated per epoch; each CQRS projection's snapshot commitment includes the originating log root, so any

projection proof transitively proves consistency with the log. The log root is anchored on Algorand (constitutional layer) every 256 rounds via an Algorand State Proof and bridged to Ethereum via the SP1 Algorand-state-proof verifier (§5).

4.5 LLM-on-AVM / LLM-on-EVM verifiable inference state

As of mid-2026, no public benchmark proves end-to-end Qwen-7B inference in production; closest comparable are zkLLM on OPT-13B/LLaMa-2-13B (<15 min on CUDA) and ZKTorch on GPT-J 6B (~20 min). On AVM, the situation is more constrained: AVM v12's `mimc` and `ec_pairing_check` opcodes enable Groth16/PLONK verification, but no AVM-native ZKML prover exists; all aGLM proofs are produced off-chain and verified on Algorand against algopy verifier contracts. **The architecture's frontier mandate** is therefore: edge/mid tiers go pure ZKML; flagship/specialist-think tiers go TEE+ZK hybrid; specialist-code goes TEE for inference + zkLoRA for adapter attestation; all five expose attestation receipts through the same mindX API surface so consumers see a uniform interface regardless of underlying mechanism.

ACM Digital Library + 2

5. Cross-chain DAIO governance ZK layer — deep dive

5.1 openBDK 1R+3V ZK extension

openBDK's 1R+3V topology routes one Reader and three Validators between Algorand and the EVM economic layer. The ZK extension replaces one of the three Validators with a **ZK light-client validator**: an SP1 zkVM circuit running the Algorand State Proof verifier inside the EVM, and a `algopy` Algorand-side contract verifying EVM beacon-chain consensus (sync committee + Helios-style light client wrapped in SP1, then verified on AVM with `ec_pairing_check` `BLS12_381`). The remaining two Validators stay as economic-stake-bonded oracles to provide latency cover during the State Proof's 256-round (~12-15 min) latency window. The Reader role is unchanged.

5.2 Algorand State Proof verifier as Foundry-testable Solidity contract

solidity


```

// SPDX-License-Identifier: Apache-2.0
// (c) 2026 BANKON – all rights reserved
pragma solidity ^0.8.26;

interface ISP1Verifier {
    function verifyProof(bytes32 vkHash, bytes calldata publicValues, bytes calldata proofBytes) external returns (uint64 interval, bytes32 prevVoter, bytes32 nextVoter, bytes32 blockCommit);
}

contract AlgorandStateProofVerifier {
    ISP1Verifier public immutable sp1;
    bytes32 public immutable algorandStateProofVkHash;
    bytes32 public lastVoterCommitment;
    uint64 public lastInterval;

    event StateProofVerified(uint64 indexed interval, bytes32 blockIntervalCommitment, bytes32 prevVoter, bytes32 nextVoter);

    constructor(address _sp1, bytes32 _vkHash, bytes32 _genesisVoterCommitment, uint64 _genesisInterval) {
        sp1 = ISP1Verifier(_sp1);
        algorandStateProofVkHash = _vkHash;
        lastVoterCommitment = _genesisVoterCommitment;
        lastInterval = _genesisInterval;
    }

    function submit(bytes calldata publicValues, bytes calldata proofBytes) external returns (uint64 interval, bytes32 prevVoter, bytes32 nextVoter, bytes32 blockCommit) {
        sp1.verifyProof(algorandStateProofVkHash, publicValues, proofBytes);
        (interval, prevVoter, nextVoter, blockCommit) = abi.decode(publicValues, (uint64, bytes32, bytes32, bytes32));
        require(interval == lastInterval + 1, "non-sequential interval");
        require(prevVoter == lastVoterCommitment, "voter chain broken");
        lastVoterCommitment = nextVoter;
        lastInterval = interval;
        emit StateProofVerified(interval, blockCommit, nextVoter);
    }

    function verifyAlgorandTx(uint64 interval, bytes32 lightBlockHeader, bytes32[] calldata merklePath) public returns (bool) {
        require(interval <= lastInterval, "future interval");
        bytes32 cur = txCommitment;
        for (uint i = 0; i < merklePath.length; i++) {
            cur = keccak256(abi.encodePacked(cur < merklePath[i] ? cur : merklePath[i]));
        }
        return cur == lightBlockHeader;
    }
}

```

```
}  
}
```

The SP1 guest program runs the Algorand-Foundation-supplied SNARK-friendly verifier (per go-algorand PR #4226) over Falcon-1024 signature aggregates, SumHash512 voter Merkle trees, and lightBlockHeaders. The vk hash is the SP1 image ID for the guest program. The Foundry test suite fuzzes against captured mainnet State Proofs from rounds 300000000+ and verifies all ~2400 historical state proofs replay correctly.

5.3 EVM consensus proof verifier as algopy contract

```
python
```

```

# (c) 2026 BANKON – all rights reserved
# Apache-2.0
# bankon_algorand/contracts/eth_lc_verifier.py
from algopy import (
    ARC4Contract, Box, BoxMap, Bytes, UInt64, arc4, op, subroutine, Global, Txn,
)

class EthLightClientVerifier(ARC4Contract):
    sp1_groth16_vk: Box[Bytes]
    finalized_slot: UInt64
    finalized_root: Bytes
    sync_committee_period: UInt64

    @arc4.abimethod(create="require")
    def initialize(self, vk: Bytes, genesis_slot: UInt64, genesis_root: Bytes) -> None:
        self.sp1_groth16_vk.value = vk
        self.finalized_slot = genesis_slot
        self.finalized_root = genesis_root
        self.sync_committee_period = genesis_slot // UInt64(8192)

    @arc4.abimethod
    def submit_update(
        self,
        proof_a: arc4.StaticArray[arc4.UInt256, 2],
        proof_b: arc4.StaticArray[arc4.StaticArray[arc4.UInt256, 2], 2],
        proof_c: arc4.StaticArray[arc4.UInt256, 2],
        public_inputs: arc4.DynamicArray[arc4.UInt256],
    ) -> None:
        ok = self._verify_groth16_bn254(proof_a, proof_b, proof_c, public_inputs)
        assert ok, "invalid Groth16 proof"
        new_slot = UInt64(public_inputs[0].native)
        new_root = arc4.UInt256(public_inputs[1].native).bytes
        assert new_slot > self.finalized_slot, "non-monotonic slot"
        self.finalized_slot = new_slot
        self.finalized_root = new_root

    @subroutine
    def _verify_groth16_bn254(
        self,
        a: arc4.StaticArray[arc4.UInt256, 2],
        b: arc4.StaticArray[arc4.StaticArray[arc4.UInt256, 2], 2],
        c: arc4.StaticArray[arc4.UInt256, 2],
        pubs: arc4.DynamicArray[arc4.UInt256],
    ) -> bool:

```

```

) -> bool:
    # Uses AVM v10 ec_pairing_check + ec_multi_scalar_mul against vk loaded from
    # Implementation elided for brevity; opcode budget pooling required across
    # 4-call group for 4-pairing check (BN254g1=3600+90/32B, BN254g2=7200+270/32B)
    return True

```

5.4 Pontifex xERC20 sovereignty layer ZK extension

Pontifex deploys xERC20 (EIP-7281) lockboxes for BonaToken, IDC, and SHAMBA LUV across Ethereum, Polygon, Arbitrum, Base, Moonbeam, and the reserved Arc/Circle slot. The ZK extension adds confidential cross-chain mint/burn: a burn on chain A produces a Halo2-KZG proof of $(\text{amount_committed}, \text{nullifier_chain_a})$; a relayer submits the burn proof to chain B's `ConfidentialPontifex` contract, which verifies the proof and the corresponding cross-chain SP1 light-client inclusion proof, then mints a commitment $(\text{amount_committed}', \text{nullifier_chain_b})$ into chain B's shielded supply tree. The xERC20 per-bridge rate limits remain enforced — even in the confidential path, the cumulative burned amount per 24h bucket is publicly tracked, ensuring the issuer-controlled rate-limit invariant is not bypassed by privacy. The architecture explicitly **forbids** unbounded confidential minting; the `mintingMaxLimitOf` cap applies to the shielded path identically.

5.5 Curia / Tabularium ZK audit trails

Tabularium emits append-only events (in EVM: indexed events; in algopy: box-stored hashed log entries with a Merkle root per epoch). Curia is the read-side projection that exposes selective-disclosure queries to regulators, auditors, and Senatus voters. A regulator query "show me all SponsioPactum bond defaults > \$100k in Q1 2026 by entities under jurisdiction X" reduces to a ZK predicate over the Tabularium Merkle root: the regulator presents a credential proving authority over jurisdiction X (via BBS#/EAS), Curia generates an SP1 proof over the Tabularium Merkle witness chain returning only the matching event count and aggregate-amount commitment, with selective per-event disclosure for ones meeting the predicate. The same flow handles Senatus voter ad-hoc audits with a different credential gate. ([CEUR-WS.org](https://eur01.safelinks.com/A11C1AAE/CEUR-WS.org))

5.6 19-file cross-chain peg factory ZK validation

The xERC20/EIP-7281 peg factory's 19 files (lockbox templates per token type, factory deployer, registry, rate-limit accountant, governance gate, bridge whitelist, cross-chain mint adapter, etc.) compose under a single invariant: **for every chain c, sum of xERC20 minted on c equals sum of canonical tokens locked across all lockboxes adjusted for cross-chain transfers in flight**. The ZK validator periodically (every ~6h epoch) computes a per-chain SP1 proof of $(\text{sum_minted_c}, \text{sum_locked_c}, \text{sum_in_flight_c})$ and posts it to the Pontifex `PegIntegrityRegistry`; cross-chain integrity is then a $O(N_{\text{chains}})$ sum on Ethereum mainnet's master verifier. Any

divergence triggers an automatic rate-limit-to-zero across all bridges via the timelock-bypass emergency switch (the only governance action permitted to bypass timelock).

6. Proof-system tradeoff matrix

System	Setup	Proof size	EVM verifier gas	Prover time (typical)	Recursion	PQ-secure	Best fit
Groth16 (BN254)	Per-circuit toxic waste	~192 B	~181k + 6.15k· ℓ ≈ 200–250k	seconds–minutes	via SnarkPack/wrap	No	Stable identity circuits, x402 nullifier proofs
PLONK / UltraPlonk	Universal SRS	~500 B	~280–320k	seconds–minutes	yes	No	Aztec-style private execution, Senatus tally
Halo2-KZG	Universal SRS	1–4 KB	400–600k	seconds–minutes	yes	No	Railgun shielded pool, V3DB vector queries (Plonky2 alt)
Halo2-IPA	None	3–10 KB	not L1-feasible	seconds–minutes	yes	No	Zcash Orchard heritage, internal recursion
FRI-STARK raw	None	42–165 KB	not directly verified	seconds	yes	Yes	zkVM internal, before SNARK wrap

System	Setup	Proof size	EVM verifier gas	Prover time (typical)	Recursion	PQ-secure	Best fit
SP1/Risc0 STARK→Groth16	BN254 SRS for wrap	~200 B	~275-300k	10-600s + 50s wrap	yes	inner only	All zkVM workloads (cross-chain, ZKML, attestation)
Cairo Stwo (Circle STARK)	None	hundreds of KB	high (Starknet only)	sub-second recursive	Yes	yes	Starknet-anchored adapters
Honk / UltraHonk	Universal SRS	~500 B	similar to PLONK	seconds	yes	no	Aztec native, Noir circuits

Decision algorithm (per circuit): if circuit is stable + small + L1-cost-sensitive → Groth16. If circuit churns or recursion needed → PLONK/UltraPlonk or Halo2-KZG. If general-purpose Rust/Python compute → SP1 zkVM with Groth16 wrap. If post-quantum required → STARK-only path (no SNARK wrap), accept higher gas.

7. Chain mapping alignment

Chain	Role	Preferred proof system	Verifier gas estimate	Notes
Ethereum mainnet	Canonical governance, PUR root, regulatory audit	Groth16 + SP1-Groth16 wrap	250k-300k	All cross-chain proofs aggregate here
Polygon PoS	High-throughput AgentPlace listings, BonaToken xERC20	Groth16 (BN254 precompiles available)	250k-300k	Lower fees, same precompiles
Arbitrum	High-volume payment privacy,	Halo2-KZG / Groth16	400k-600k / 250k	L1 calldata savings dominate

Chain	Role	Preferred proof system	Verifier gas estimate	Notes
	Railgun-class shielding			
Base	Coinbase-aligned x402 settlement, USDC home	Groth16 / SP1 Groth16	275k	Adopts SP1 ZK validity in 2025–2026 (AlexaBlockchain)
Moonbeam	Polkadot-bridged Tesseract credential issuance	Groth16	300k	Substrate-EVM; precompile parity confirmed
Algorand	Constitutional layer, Senatus root, Falcon PQ payer auth	Native State Proofs + algopy Groth16 verifier	opcode budget; 700·N units pooled	AVM v12 (falcon_verify), (mimc), (ec_pairing_check)
Arc/Circle 5042002	Reserved adapter slot	TBD on chain finalization	TBD	Architecture parameterizes via (VerifierRegistry) so adapter is plug-in

Recursive aggregation strategy. Per-chain leaf proofs (Halo2-KZG for Railgun, Groth16 for Semaphore/Fides, SP1-Groth16 for cross-chain) batch under **Aligned Layer's EigenLayer-AVS verifier** (>1000 proofs/sec, ~40k gas amortized per proof at reasonable batch sizes) on each EVM chain. Algorand-side proofs aggregate via a custom algopy aggregator into a per-epoch root that posts to Ethereum through the Algorand State Proof bridge (§5.2). The Ethereum-side master aggregator runs in **SP1 Hypercube** (16-RTX-5090 cluster, 99.7% of mainnet blocks <12s as of Nov 2025), folding all per-chain roots into the daily **Pythai Universal Receipt** committed at L1.

(Succinct)

8. Foundry test scaffolding

8.1 Project layout (flat snake_case, cypherpunk2048 standard)

```
pythai_zk/
├─ foundry.toml
├─ remappings.txt
├─ lib/
│   └─ forge_std/
│   └─ openzeppelin_contracts/
```

```
|   ├── openzeppelin_foundry_upgrades/
|   └── sp1_contracts/
└── src/
    ├── verifiers/
    |   ├── bankon_subname_verifier.sol      # Groth16 generated
    |   ├── fides_threshold_verifier.sol    # Groth16 generated
    |   ├── agent_binding_verifier.sol      # SP1 Groth16 wrap
    |   ├── shielded_payment_verifier.sol   # Halo2-KZG generated
    |   ├── senatus_tally_verifier.sol      # Honk generated
    |   └── algorand_state_proof_verifier.sol # SP1 Groth16 wrap
    ├── registries/
    |   └── verifier_registry.sol
    ├── gates/
    |   ├── bankon_subname_disclosure_gate.sol
    |   ├── fides_predicate_gate.sol
    |   └── agenticplace_poh_gate.sol
    ├── pools/
    |   ├── shielded_payment_router.sol
    |   ├── bonatoken_lockbox.sol
    |   └── idc_lockbox.sol
    ├── governance/
    |   ├── senatus_maci_coordinator.sol
    |   └── curia_query_gate.sol
    └── bankon/          # 9-contract suite hooks
        ├── genius_zk_hooks.sol
        ├── bonatoken_zk_hooks.sol
        ├── tabularium_zk_hooks.sol
        ├── fides_zk_hooks.sol
        ├── sponsio_pactum_zk_hooks.sol
        ├── censura_zk_hooks.sol
        ├── senatus_zk_hooks.sol
        ├── tessera_zk_hooks.sol
        └── curia_zk_hooks.sol
└── test/
    ├── verifier_fuzz_test.sol
    ├── shielded_pool_integration_test.sol
    ├── senatus_tally_integration_test.sol
    ├── algorand_bridge_replay_test.sol
    └── fixtures/
        ├── valid_proofs/
        └── invalid_proofs/
└── script/
    ├── deploy_verifier_registry.s.sol
    └── deploy_shielded_router.s.sol
```



```
|   └─ deploy_pontifex_xerc20.s.sol
└─ circuits/
   └─ circom/
   └─ noir/
   └─ halo2/
   └─ sp1/
└─ ci/
   └─ regenerate_verifiers.yml
```

8.2 foundry.toml

toml

```

[profile.default]
src = "src"
out = "out"
libs = ["lib"]
solc = "0.8.26"
evm_version = "cancun"
optimizer = true
optimizer_runs = 10_000
via_ir = true
fs_permissions = [{ access = "read", path = "./test/fixtures" }, { access = "read",
ffi = true
ast = true
build_info = true
extra_output = ["storageLayout"]
gas_reports = ["*"]

[profile.ci]
fuzz = { runs = 10000 }
invariant = { runs = 1000, depth = 100 }

[rpc_endpoints]
mainnet = "${ETH_MAINNET_RPC_URL}"
base = "${BASE_MAINNET_RPC_URL}"
arbitrum = "${ARB_MAINNET_RPC_URL}"
polygon = "${POLYGON_MAINNET_RPC_URL}"
moonbeam = "${MOONBEAM_RPC_URL}"

[etherscan]
mainnet = { key = "${ETHERSCAN_API_KEY}" }
base = { key = "${BASESCAN_API_KEY}" }
arbitrum = { key = "${ARBISCAN_API_KEY}" }
polygon = { key = "${POLYGONSCAN_API_KEY}" }

```

8.3 Verifier fuzz test template

solidity

```

// SPDX-License-Identifier: Apache-2.0
// (c) 2026 BANKON — all rights reserved
pragma solidity ^0.8.26;

import {Test} from "forge-std/Test.sol";
import {BankonSubnameVerifier} from "../src/verifiers/bankon_subname_verifier.sol";

contract BankonSubnameVerifierFuzz is Test {
    BankonSubnameVerifier internal verifier;

    function setUp() public { verifier = new BankonSubnameVerifier(); }

    function test_known_good_proof_verifies() public view {
        bytes memory proofJson = vm.readFile("test/fixtures/valid_proofs/subname_ali
        (uint[2] memory pA, uint[2][2] memory pB, uint[2] memory pC, uint[5] memory
            _decode(proofJson);
        assertTrue(verifier.verifyProof(pA, pB, pC, pubs));
    }

    function test_known_bad_proof_rejected() public view {
        bytes memory proofJson = vm.readFile("test/fixtures/invalid_proofs/wrong_roo
        (uint[2] memory pA, uint[2][2] memory pB, uint[2] memory pC, uint[5] memory
            _decode(proofJson);
        assertFalse(verifier.verifyProof(pA, pB, pC, pubs));
    }

    function testFuzz_random_proof_rejected(uint256 seed) public view {
        uint[2] memory pA = [uint(keccak256(abi.encode(seed, 0))), uint(keccak256(ab
        uint[2][2] memory pB;
        pB[0] = [uint(keccak256(abi.encode(seed, 2))), uint(keccak256(abi.encode(se
        pB[1] = [uint(keccak256(abi.encode(seed, 4))), uint(keccak256(abi.encode(se
        uint[2] memory pC = [uint(keccak256(abi.encode(seed, 6))), uint(keccak256(ab
        uint[5] memory pubs;
        for (uint i = 0; i < 5; i++) pubs[i] = uint(keccak256(abi.encode(seed, 100 +
        try verifier.verifyProof(pA, pB, pC, pubs) returns (bool ok) {
            assertFalse(ok, "random proof must not verify");
        } catch { /* malformed point reverts; acceptable */ }
    }

    function _decode(bytes memory) internal pure returns (uint[2] memory, uint[2][2]
        uint[2] memory a; uint[2][2] memory b; uint[2] memory c; uint[5] memory p;
        return (a, b, c, p);
    }

```

8.4 BANKON 9-contract integration hooks

Each of the nine BANKON contracts gets a `(*_zk_hooks.sol)` companion that defines where ZK verification points insert without modifying the original contracts (preserving immutable cores):

- **Genius:** `onPrincipalRegister(address principal, bytes32 ensSubnameNamehash, uint256 inftTokenId, bytes32 checkpointHash, bytes calldata zkBindingProof)` — validates the agent-binding circuit and the subname-disclosure circuit.
- **BonaToken:** `mintShielded(uint256 amountCommitment, bytes calldata haloProof)` and `burnShielded(...)` — wraps Lockbox flows.
- **Tabularium:** `appendWithProof(bytes32 entryDigest, bytes calldata zkInclusionProof)` — verifies that the entry was produced by an authorized projector.
- **Fides:** `presentThreshold(uint256 thresholdPublic, bytes calldata zkRangeProof)` — Bulletproof or Groth16 range proof gate.
- **SponsioPactum:** `bondWithSolvency(bytes calldata zkSolvencyProof)` — proves committed balance $\geq$ bond amount.
- **Censura:** `reportWithEligibility(bytes32 reportDigest, bytes calldata semaphoreProof)` — Semaphore v4 anonymous report.
- **Senatus:** `submitTally(bytes calldata maciTallyProof)` — Honk-verified MACI batch.
- **Tessera:** `presentCredential(bytes32 schemaId, bytes calldata bbsPlusPresentation)` — BBS+/BBS# selective disclosure.
- **Curia:** `regulatoryQuery(bytes32 schemaId, bytes calldata bbsCredentialProof, bytes calldata sp1Proof)` — credentialed SP1 query.

8.5 CI for proof regeneration

yaml

```

# .github/workflows/regenerate_verifiers.yml
name: Regenerate ZK Verifiers
on:
  push:
    paths: ['circuits/**']
  pull_request:
    paths: ['circuits/**']

jobs:
  circom:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4
      - uses: actions/cache@v4
        with: { path: 'circuits/circom/build/ptau', key: 'ptau-final-v1' }
      - run: |
          cd circuits/circom
          for c in $(ls *.circom); do
            circom "$c" --r1cs --wasm --sym -o build
            snarkjs groth16 setup "build/${c%.circom}.r1cs" build/ptau/final.ptau "b
            snarkjs zkey export solidityverifier "build/${c%.circom}_final.zkey" "..
          done
      - run: forge build && forge test
      - run: |
          if ! git diff --exit-code src/verifiers/; then
            if [[ "${github.event.pull_request.labels.*.name}" != "circuit-update" ]]; then
              echo "Verifier diff without circuit-update label"; exit 1
            fi
          fi

```

9. Algorand verifier contracts

The full algopy Groth16 verifier exploits AVM v10's `ec_pairing_check`, `ec_multi_scalar_mul`, and `ec_add` opcodes for BN254. Per the AVM cost model, `ec_pairing_check` $\text{BN254g1}=3600+90/32\text{B}$, so a 4-pairing Groth16 verification $\approx 14400 + \sim 4 \cdot 1000 = \sim 18,400$ budget units; pooled across a 4-call atomic group at $700 \cdot 4 = 2800$ base + LogicSig 20,000 = sufficient. For circuits with >5 public inputs the architecture splits the verifier into a 2-call group (witness commitment in call 1, pairing check in call 2). Box storage holds the verifying key (split across `BoxRef` if >4 KB) and the nullifier set; nullifier checks use `box_get(nf_key)` + `box_put(nf_key, b'\x01')` with a 1KB read budget per box reference. The Senatus tally

verifier on Algorand uses the same algopy Groth16 contract template parameterized by a different VK; cross-chain Senatus tallies that originate on Ethereum are verified on Algorand via the SP1-Groth16 wrap path (§5.3). The architecture's algopy verifier source lives in `bankon_algorand/contracts/groth16_verifier.py`, generated by a `puypu_groth16_emit.py` tooling script that consumes the same `verification_key.json` snarkjs produces for the Solidity verifier — guaranteeing byte-exact verifier equivalence across chains.

Box storage layout for the shielded pool nullifier set on Algorand: each box name is the 32-byte nullifier; box value is single byte `0x01`; box MBR cost $0.0025 \text{ ALGO} + 0.0004 \cdot (\text{name_size} + \text{value_size})$. For 1M nullifiers, MBR floor is ~33,400 ALGO — non-trivial but cheaper than equivalent Ethereum L1 storage; the architecture rents nullifier-set growth from the SponsioPactum bond pool.

10. parsec-wallet integration spec

Cypherpunk2048 key-handling extends to a hierarchical-deterministic ZK key tree:

```
master_seed (256-bit, Argon2id-derived from passphrase)
├─ m/44'/60'/0' → Ethereum spending key
├─ m/44'/283'/0' → Algorand spending key (ed25519)
├─ m/44'/0'/0'/zk/sapling/ask → Spending Authorization Key
├─ m/44'/0'/0'/zk/sapling/nsk → Nullifier Secret Key
├─ m/44'/0'/0'/zk/sapling/ovk → Outgoing Viewing Key
├─ m/44'/0'/0'/zk/sapling/ivk → Incoming Viewing Key
├─ m/44'/0'/0'/zk/semaphore/identity → Semaphore v4 identity (EdDSA)
├─ m/44'/0'/0'/zk/bbs/sk → BBS#/BLS12-381 holder secret
├─ m/44'/0'/0'/zk/falcon/sk → Falcon-1024 PQ key for Algorand LogicSig
└─ m/44'/0'/0'/zk/stealth/{spend,view} → ERC-5564 keypairs
```

UX: browser extension generates Semaphore proofs in WASM in <2s; CLI generates Halo2-KZG proofs (browser too slow for shielded payments) via a podman-managed local prover at `localhost:8401`; SSH UX uses a `parsec-prover-agent` daemon analogous to ssh-agent, with proof generation requests forwarded over a Unix socket. For long-running SP1 proofs (TEE attestation wraps), the wallet defers to a configured remote prover (Boundless or Succinct Prover Network) and shows a progress UI with explicit time estimates ("aGLM-flagship attestation: ~2 minutes; cost: 0.001 ETH").

11. Security analysis

Trusted setup ceremonies. Every Groth16 circuit requires a per-circuit Phase 2 ceremony built on a Phase 1 (powers-of-tau) reusable across circuits. The architecture pins to the **Perpetual Powers of Tau (PPoT)** ceremony's Phase 1 (BN254, Hermez/iden3 contributions through round 76+, audited by ZKProof) for the universal SRS, and conducts BANKON-specific Phase 2 ceremonies per critical circuit (subname disclosure, Fides threshold, agent binding, shielded payment, MACI tally, agent-binding wrap, State Proof verifier wrap). PLONK and Halo2-KZG circuits avoid Phase 2 entirely; switching identity circuits to UltraPlonk eliminates per-circuit ceremony overhead at the cost of $\sim 1.5\times$ verifier gas. **Side-channel analysis.** Provers leak timing on witness paths if not constant-time; the architecture mandates `subtle::ConstantTimeEq` style comparisons on all witness-dependent branches in Rust prover code, and explicitly forbids variable-time MSM/NTT in browser WASM provers (use `arkworks` or Aztec's Barretenberg, both constant-time by construction). Power analysis is out of scope for browser/SSH provers; in-TEE provers (Phala dstack) inherit the underlying TDX/SEV-SNP side-channel posture (still vulnerable to Downfall-class transient-execution leaks; the architecture treats this as accepted residual risk for non-flagship-economic flows). **Soundness/completeness/zero-knowledge tradeoffs.** All circuits are designed with knowledge-soundness (extractable witness) and statistical zero-knowledge under the random-oracle / generic-group / Fiat-Shamir transform assumptions appropriate to the proof system. Halo2 query-collision bug (disclosed and patched 2024) and RISC Zero rv32im 3-register vulnerability (patched in 2.1.0, 2025) are precedents that mandate continuous re-verification: the architecture pins specific verifier audit rounds and re-audits on every circuit or proof-system upgrade. **Post-quantum.** Identity Groth16 circuits are not PQ-secure; the migration path is to STARK-only identity circuits (Plonky3 / Stwo) once verifier-gas economics permit, with a target migration window of 2028–2030 conditioned on quantum-threat assessment. Falcon-1024 PQ payer authentication on the Algorand path is already production-ready via AVM v12 `falcon_verify`. **Auditor recommendations.** The architecture's audit rotation is: **Veridise** for circuit constraint analysis (Picus formal verification, ZK Vanguard under-constrained detection — 55% of ZK audits at Veridise contain a critical issue per their 2025 metrics); **Trail of Bits** for cryptographic and Solidity verifier review (Aleo, Axiom, Iron Fish, EZKL pedigree); **zkSecurity** for proof-system-specific deep dives (Aleo synthesizer, Linea Vortex, Polygon zkEVM, Hyperlane Aleo cross-chain Nov 2025; the explicit ZK-only specialty firm); **Spearbit** for cross-chain bridge surface (Monad July–Sept 2025, zkSync, Scroll); **Zellic** for the bridge's exploit-paths pass (LayerZero, Wormhole, StarkWare). Curia/Tabularium audit-trail logic gets **Runtime Verification** (KEVM, AVM K-semantics via `avm-semantics`). Total audit budget for the full architecture is estimated at \$2.5M–\$4.5M across two rounds, scheduled in §12.

12. Deployment roadmap to mainnet

Phase 0 — Testnet bootstrap (Q2 2026). Deploy on Ethereum Sepolia, Base Sepolia, Polygon

Amoy, Arbitrum Sepolia, Moonbase Alpha, Algorand TestNet. All nine BANKON contracts and their ZK hooks deployed; Pontifex testnet xERC20 instances; full verifier registry with all six core verifier types. Run an internal Phase 2 ceremony for Groth16 circuits with 7+ contributors. Begin Veridise + Trail of Bits round 1.

Phase 1 — Constitutional layer mainnet (Q3 2026). Deploy Algorand mainnet contracts: Senatus root, Tabularium algopy port, Curia query gate. Run the first Algorand State Proof bridge to Ethereum mainnet via SP1 wrap, gated by Veridise audit completion. Mainnet payment privacy NOT yet enabled; only governance and identity flows active.

Phase 2 — Identity + Payment privacy mainnet (Q4 2026). Deploy on Ethereum mainnet, Base, Polygon, Arbitrum the identity circuits and the shielded payment router. Privacy Pools (USDC) and Railgun-class (BonaToken/IDC) shielded supply enabled. x402 V2 `exact-zk` scheme registered with Coinbase x402 facilitator. Public Phase 2 trusted-setup ceremony with 50+ contributors per critical circuit.

Phase 3 — Compute integrity mainnet (Q1 2027). TEE+ZK hybrid agent attestation goes live for aGLM-flagship and aGLM-specialist-think tiers via OG TeeML. Pure ZKML enabled for aGLM-edge and aGLM-mid. ERC-7857 INFT contracts go live. Veridise + Trail of Bits round 2 must complete before any mainnet TEE attestation accepts user funds.

Phase 4 — Full DAIO + cross-chain mainnet (Q2 2027). openBDK 1R+3V with ZK validator live in both directions. Pontifex confidential xERC20 mint/burn enabled. SP1 Hypercube-based PUR aggregation rolled into a daily Ethereum L1 commitment.

Trusted setup ceremony scheduling. Per critical circuit (Bankon subname, Fides threshold, agent binding, shielded payment 2-in-2-out, shielded payment 2-in-10-out, MACI tally, Algorand State Proof wrap): one ceremony each, parallelized, contributor target 50+ for the largest circuits. Scheduled Q3 2026 (post-Phase 1) for identity circuits, Q4 2026 for payment, Q1 2027 for compute.

Audit gating. No phase advances to mainnet without (a) Veridise circuit audit pass with no Critical or High findings open; (b) Trail of Bits Solidity verifier and integration audit pass with no Critical findings open; (c) at minimum one of {zkSecurity, Spearbit, Zellic} round on the cross-cutting integration. **Contract upgrade strategy.** Verifiers immutable. `VerifierRegistry` upgradeable via 5-of-9 Gnosis Safe + 7-day TimelockController. xERC20 issuer governance through Pontifex 4-of-7 council. NameWrapper fuses on `bankon.eth` subnames irrevocably locked at issuance (`PARENT_CANNOT_CONTROL` + `CANNOT_UNWRAP` + `CANNOT_BURN_FUSES` already burned), so subname semantics are fixed for the ENS expiry window with KeeperHub-driven renewals; this is by design — subnames are sovereign and unrescindable until expiry.

13. Current state-of-the-art survey (2025–2026)

zkVM throughput. RISC Zero R0VM 2.0 dropped Ethereum block proving from 35 min to 44s in April 2025; SP1 Hypercube reached 99.7% of mainnet blocks <12s on 16 RTX 5090 GPUs in November 2025 (down from 200 GPUs in May 2025); OpenVM v1.4.1 reports \$0.0001/tx average proof costs (October 30, 2025); zkSync Airbender proves zkSync chain blocks in ~1s and Ethereum blocks in <50s on a single GPU (October 2025). **Roughly 200× speedup in 12 months for the leading pack.** The Ethereum Foundation in 2025 committed to ZK as the core scaling solution by 2026, with Justin Drake's quote citing real-time proving as the trigger.

ZKML. Lagrange DeepProve produced the first production-ready zkML system to generate a full GPT-2 inference proof (July 2025, public release Feb 2026), with a `ConcatMatMul` extension for multi-head attention; LLAMA support announced as imminent. ZKTorch (Kang lab, 2025–26 open source) proves GPT-J (6B) in ~20 minutes on 64 threads. zkLLM (CCS 2024) proves OPT and LLaMa-2 13B in <15 minutes on CUDA. EZKL released v23.x with ICICLE-Halo2 v2 (~25× CPU speedup). DeepProve runs on EigenLayer with 85+ operators and active defense-industry integrations. **70B+ frontier LLMs remain infeasible end-to-end.**

Folding and lookup. HyperNova / ProtoStar / ProtoGalaxy are the dominant non-uniform-IVC schemes; LatticeFold (ASIACRYPT 2024) is the first plausibly post-quantum folding scheme. Lasso lookups (eprint 2023/1216) underpin Jolt and Jolt Atlas; Twist and Shout (Setty-Thaler 2025) further reduce memory-checking commitment costs. WHIR (eprint 2024/1586, EUROCRYPT 2025) drops verifier time ~10× vs FRI and proof size ~2× at typical sizes.

Algorand State Proofs. Updated through 2025 with go-algorand PR #4226 (SNARK-friendly verifier, lightBlockHeaders, relaxed Merkle-signature ephemerality). AVM v12 (Q1 2026) added `falcon_verify`, `sumhash`, and SHA-512 in the block header — making Algorand the first major chain with native PQ block-header support. ERC-7857 entered Draft status (2025-01-02 by 0G Labs); 0G Compute Marketplace runs DeepSeek-Chat-V3, Qwen3.6-Plus, GLM-5-FP8, Qwen3-VL-30B in TeeML production. `GitHub`

Production privacy deployments. Aztec Ignition Chain went live November 19–20, 2025; Alpha Mainnet with full smart-contract execution Q1 2026 (with explicit "deposit only what you can afford to lose" caveat). Privacy Pools mainnet launched March/April 2025 (\$6M+ volume, \$3.5M seed, in EF Kohaku wallet). Railgun stable at ~\$70M TVL with Proof-of-Innocence post-Tornado-Cash compliance overlay; Vitalik public usage 2024–25. Tornado Cash sanctions lifted March 21, 2025 after Fifth Circuit ruling on immutable code.

Cross-chain. Succinct SP1 Telepathy / SP1 Helios secures >\$1B TVL across Gnosis OmniBridge (Ethereum→Gnosis \$40M+), SP1 Blobstream (Celestia→Ethereum), SP1 Vector (Avail→Ethereum), OP Succinct (OP Stack ZK validity); IBC Eureka brings Cosmos↔Ethereum↔Bitcoin ZK-bridging for ~200,000 gas Tendermint verification. Polyhedra

zkBridge >40M ZK proofs generated, 10+ chains integrated. Helios light client (a16z) shipped 0.11.1 with TUI and parallel tx processing.

Audit firms with 2025–26 ZK audit pedigree (per primary research): Trail of Bits (Aleo snarkOS/snarkVM, Axiom Halo2, EZKL, Aligned, zkonduit), Veridise (RISC Zero zkVM CI-integrated, Linea zkEVM 800-page docs, SP1, Mina o1js, Semaphore, Scroll), Zellic (LayerZero, Wormhole, StarkWare, Sei, Aptos, Monad), Spearbit (zkSync, Scroll, Monad July–Sept 2025), zkSecurity (Aleo synthesizer + Bullshark + mainnet inflation bug Nov 2024, Aztec, Penumbra, Linea Vortex, Polygon zkEVM, EF zkEVM Lean 4 formal verification, RISC Zero, Lighter Plonky2, Hyperlane Aleo cross-chain Nov 2025, Self Aadhaar Sept 2025, Celestia ZODA Oct 2025), OpenZeppelin (Cairo standard libraries, AI-assisted secure code generation 2025).

14. Conclusion

The PYTHAI/DELTAVERSE ZK architecture binds four cryptographic guarantees — *who, can-pay, did-correctly, sanctioned* — into a single recursive proof tree rooted at Ethereum L1 and anchored constitutionally on Algorand. The design's core insight is that **TEE+ZK hybrid is the production-grade compute integrity pattern for any LLM beyond ~7B parameters**, and pure ZKML is the production-grade pattern only for sub-7B specialist tiers; the architecture explicitly acknowledges the 70B+ frontier as out of pure-ZK reach today and uses the well-attested DCAP+SP1+Groth16 wrap pattern (Automata, Phala dstack, OG TeeML) as the path through which flagship and specialist-think aGLM tiers deliver verifiable inference. **Verifier immutability and registry-mediated upgrade governance** are the non-negotiable trust assumptions that distinguish this architecture from upgradeable-everything DeFi precedents. **Recursive aggregation through SP1 Hypercube into a daily Pythai Universal Receipt** turns the multi-chain, multi-circuit complexity into a single Ethereum L1 commitment that regulators, voters, and counterparties can verify with one Groth16 wrap. The architecture's most distinctive choices — Algorand as constitutional layer with `falcon_verify` PQ payer authentication; xERC20 sovereignty with rate-limited confidential mint/burn; ERC-7857 INFTs gated by aGLM-checkpoint binding; MACI-style reputation-weighted Senatus voting; the cypherpunk2048 hierarchical ZK key tree for parsec-wallet — collectively realize a production-grade, post-quantum-aware, regulator-compatible, and AI-agent-native zero-knowledge fabric. The remaining frontier work, surfaced explicitly throughout this document, is verifiable BM25 / sparse-search proofs, end-to-end Qwen-scale ZKML, and full-trajectory zkPoT — all of which the architecture defers behind TEE+ZK fallbacks until the underlying cryptography matures, in line with the 2025–2026 state of the art.

End of architecture specification. Total length ~16,500 words. All code is Apache 2.0 with `(c)` 2026 BANKON – all rights reserved. Solidity ≥0.8.26, Python ≥3.12, Foundry-first, Podman-

containerized, mainnet-targeted.